

Previsão de Fluxo de Tráfego Rodoviário

Design e Otimização de Modelos de Machine Learning

Mestrado em Inteligência Artificial

Unidade Curricular:
Dados e Aprendizagem Automática

Instituição:
Universidade do Minho

Grupo de Trabalho:

Diogo Dias (PG60245)
Eduardo Vilaça (PG60252)
Nuno Ni (PG60285)

Braga,10 de Janeiro de 2026

Índice

1 Introdução 3
2 Metodologia 3
3 Business Understanding 4
3.1 Contexto e Dataset 4
3.2 Definição do Problema 4
4 Data Understanding 4
4.1 Análise e Exploração de Dados 5
4.1.1 Inspeção da Estrutura 5
4.1.2 Análise de Qualidade dos Dados 6
4.1.3 Análise Univariada (Distribuições e Variância) 7
4.1.4 Análise Multivariada e Correlações 8
5 Data Preparation 10
5.1 Tratamento de Variáveis Categóricas 10
5.1.1 Normalização da Nebulosidade 10
5.1.2 Codificação da Luminosidade 11
5.2 Engenharia de Atributos Temporais (*Feature Engineering*) 11
5.3 Seleção de Variáveis (**Feature Selection**) 12
5.4 Estruturação do Dataset e Codificação do Alvo 13
5.4.1 Tratamento da Variável Alvo (**Target Encoding**) 13
6 Modeling 14
6.1 Estratégia de Validação e Divisão de Dados 14
6.2 Análise Comparativa de Modelos 15
6.3 A nossa escolha: XGBoost 16
6.4 Otimização de Hiperparâmetros (Grid Search) 16
7 Evaluation 17
7.1 Desempenho Global 17
7.2 Análise por Classe 17
7.3 Análise Crítica 18
7.4 Correlação entre Features 19
7.5 Correlação com a variável alvo 19
8 Deployment 20
9 Conclusão 21

1 Introdução

A modelação do fluxo de tráfego rodoviário constitui um desafio clássico no domínio dos sistemas de transporte inteligentes, caracterizando-se por uma natureza inerentemente estocástica e não-linear. Numa era de urbanização crescente, a capacidade de prever congestionamentos é fundamental não só para a gestão eficiente da mobilidade urbana, mas também para a redução de emissões poluentes e otimização do planeamento civil.

Este relatório surge no âmbito da competição "ML @DAA - Edição 2025/2026", inserida na Unidade Curricular de Dados e Aprendizagem Automática (DAA) da Universidade do Minho, visando o desenvolvimento de modelos preditivos para o tráfego na cidade do Porto.

2 Metodologia

Para abordar o problema, adotou-se uma metodologia baseada no ciclo **CRISP-DM** (**Cross-Industry Standard Process for Data Mining**), estruturada em seis fases essenciais:

1. **Business Understanding:** Compreensão do contexto e definição do problema e objetivos do projeto.
2. **Data Understanding:** Recolha inicial, descrição e exploração de dados para identificar qualidade e padrões.
3. **Data Preparation:** Seleção, limpeza e transformação de dados brutos (*Feature Engineering*).
4. **Modeling:** Seleção e aplicação de técnicas de modelação (ex: *XGBoost, Random Forest* etc.) e calibração de parâmetros.
5. **Evaluation:** Avaliação rigorosa dos modelos face aos objetivos (*Accuracy e F1-Score*).
6. **Deployment:** Submissão dos resultados na plataforma da competição (*Kaggle*).

A reprodutibilidade dos resultados foi assegurada através da fixação de *seeds* (`random_state=2025`) em todas as etapas estocásticas.

3 Business Understanding

3.1 Contexto e Dataset

O estudo incide sobre a rede rodoviária da cidade do Porto, utilizando um conjunto de dados recolhido num período superior a um ano (julho de 2018 a outubro de 2019). O dataset agrega informações heterogéneas que caracterizam cada ponto temporal, incluindo métricas de fluxo livre (*free flow*), dados temporais e uma componente meteorológica detalhada (temperatura, pressão atmosférica, humidade, velocidade do vento e nebulosidade).

3.2 Definição do Problema

O objetivo central é o desenvolvimento de um modelo de classificação multiclasse supervisionada. A variável alvo (*target*), `AVERAGE_SPEED_DIFF`, representa a diferença entre:

1. A velocidade máxima teórica em cenários de fluxo livre;
2. A velocidade real verificada no ponto temporal específico.

Esta métrica é discretizada numa escala qualitativa de cinco níveis: *`None`* (trânsito inexistente), *`Low`*, *`Medium`*, *`High`* e *`Very_High`* (congestionamento severo).

4 Data Understanding

A fase de **Data Understanding** é fundamental para garantir a viabilidade do projeto. Nesta etapa, o foco incidiu na exploração profunda do dataset para compreender a natureza das variáveis, identificar falhas de integridade e extrair os primeiros ***insights*** sobre o comportamento do tráfego.

O processo estruturou-se em quatro eixos: (1) inspeção estrutural e estatística, (2) auditoria de qualidade (*missing values*), (3) análise de distribuições e (4) estudo de relações multivariadas. Esta abordagem permitiu não só detetar variáveis irrelevantes (como constantes), mas também fundamentar as estratégias de limpeza e transformação de dados que seriam aplicadas na fase seguinte.

4.1 Análise e Exploração de Dados

4.1.1 Inspeção da Estrutura

Numa primeira instância, analisaram-se os tipos de dados e a integridade das colunas utilizando, principalmente, as funções `info()` e `describe()`.

```
# Inspeção inicial da estrutura e estatísticas descritivas
df_train.info()
df_train.head(10)

print(df_train.columns)
df_train.dtypes

# Resumo estatístico das variáveis numéricas
df_train.describe()
```

```
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   city_name               6812 non-null   object
 1   record_date             6812 non-null   object
 2   AVERAGE_SPEED_DIFF      4612 non-null   object
 3   AVERAGE_FREE_FLOW_SPEED 6812 non-null   float64
 4   AVERAGE_TIME_DIFF       6812 non-null   float64
 5   AVERAGE_FREE_FLOW_TIME  6812 non-null   float64
 6   LUMINOSITY              6812 non-null   object
 7   AVERAGE_TEMPERATURE     6812 non-null   float64
 8   AVERAGE_ATMOSP_PRESSURE 6812 non-null   float64
 9   AVERAGE_HUMIDITY        6812 non-null   float64
 10  AVERAGE_WIND_SPEED      6812 non-null   float64
 11  AVERAGE_CLOUDINESS      4130 non-null   object
 12  AVERAGE_PRECIPITATION   6812 non-null   float64
 13  AVERAGE_RAIN            563 non-null    object
dtypes: float64(8), object(6)
memory usage: 745.2+ KB
```

Figura 1: Estrutura e tipos de dados das colunas (`info`).

	AVERAGE_FREE_FLOW_SPEED	AVERAGE_TIME_DIFF	AVERAGE_FREE_FLOW_TIME	AVER
count	6812.000000	6812.000000	6812.000000	
mean	40.661010	25.637111	81.143952	
std	4.119023	33.510507	8.294401	
min	30.500000	0.000000	46.400000	
25%	37.600000	2.275000	75.400000	
50%	40.700000	12.200000	82.400000	
75%	43.500000	36.200000	87.400000	
max	55.900000	296.500000	112.000000	

Figura 2: Resumo das estatísticas descritivas (`describe`).

A análise preliminar revelou que o dataset é composto maioritariamente por variáveis numéricas, mas contém colunas críticas como objetos, nomeadamente: AVERAGE_SPEED_DIFF (*target*), LUMINOSITY, AVERAGE_CLOUDINESS e AVERAGE_RAIN.

Esta observação ditou a necessidade de um mapeamento categórico ou conversão para formato numérico para que os algoritmos de Machine Learning pudessem processar a informação.

4.1.2 Análise de Qualidade dos Dados

A identificação de dados em falta (*missing values*) foi realizada através da visualização de um heatmap e contagem direta.

```
# Visualização de dados em falta
sns.heatmap(df_train.isnull(), yticklabels=False, cbar=False, cmap='viridis')

# Contagem absoluta de nulos
print(df_train.isna().sum())
```

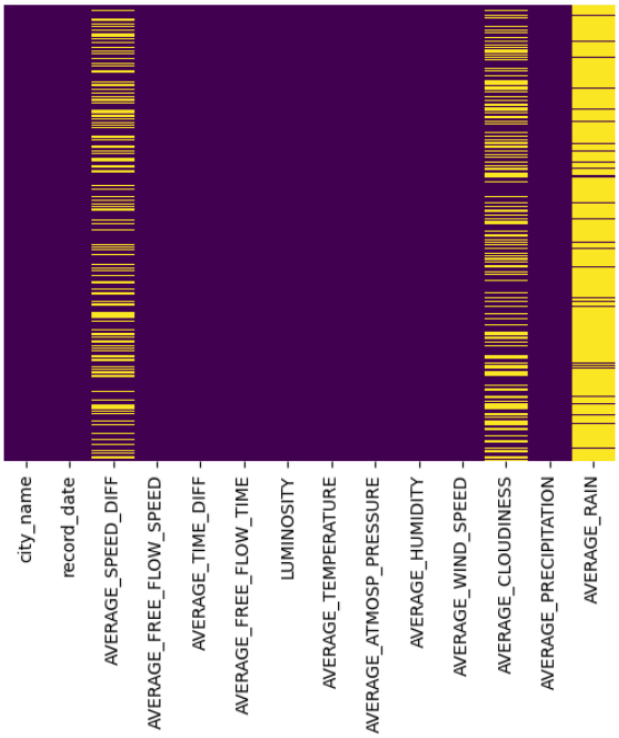


Figura 3: Mapa térmico (heatmap) de valores nulos. As barras amarelas indicam ausência de dados.

```
city_name                    0
record_date                  0
AVERAGE_SPEED_DIFF        2200
AVERAGE_FREE_FLOW_SPEED      0
AVERAGE_TIME_DIFF            0
AVERAGE_FREE_FLOW_TIME       0
LUMINOSITY                   0
AVERAGE_TEMPERATURE          0
AVERAGE_ATMOSP_PRESSURE      0
AVERAGE_HUMIDITY             0
AVERAGE_WIND_SPEED           0
AVERAGE_CLOUDINESS        2682
AVERAGE_PRECIPITATION        0
AVERAGE_RAIN              6249
dtype: int64
```

Figura 4: Contagem dos valores nulos em cada coluna.

Identificou-se assim um padrão crítico de ausência de dados:

`AVERAGE_CLOUDINESS`: Apresenta uma taxa de valores nulos de aproximadamente 40%, exigindo uma estratégia de imputação robusta (como a moda) para não descartar registos valiosos.

`AVERAGE_RAIN`: Apresenta uma taxa de omissão crítica (~90%). Esta descoberta levou o grupo a ponderar remover por completo a *feature*.

4.1.3 Análise Univariada (Distribuições e Variância)

Nesta etapa, analisou-se o equilíbrio das classes e a variabilidade das características individuais. O objetivo primordial foi identificar o "ruído" no dataset (variáveis que não possuem poder preditivo) e compreender a cardinalidade das variáveis categóricas para definir a melhor estratégia de codificação.

```
# Verificação do desequilíbrio da variável alvo
df_train['AVERAGE_SPEED_DIFF'].value_counts()

# Verificação da cardinalidade de todas as colunas
print(df_train.nunique())

# Análise de variáveis categóricas e do AVERAGE_PRECIPITATION
df_train['LUMINOSITY'].value_counts()
df_train['AVERAGE_CLOUDINESS'].value_counts()
df_train['AVERAGE_RAIN'].value_counts()
df_train['AVERAGE_PRECIPITATION'].value_counts()
```

As principais conclusões desta análise foram:

Remoção de Variáveis de Variância Zero: Através da métrica de `nunique()`, confirmou-se que as variáveis `city_name` (Porto) e `AVERAGE_PRECIPITATION (0.0)` possuem um único valor em todas as 6812 linhas do dataset de treino. Uma variável com variância zero não contém informação útil para separar as classes, pelo que foram removidas para reduzir a dimensionalidade e evitar redundância.

Estratégia de Codificação da Luminosidade: Observou-se que a variável `LUMINOSITY` possui baixa cardinalidade (3 valores únicos) e apresenta uma relação de hierarquia natural (`DARK < LOW_LIGHT < LIGHT`). Esta natureza ordinal validou a nossa opção pelo Ordinal Encoding em detrimento do One-Hot Encoding. Esta escolha permite que o modelo perceba a progressão da luminosidade sem expandir o número de colunas do dataset.

Limpeza Semântica e Gestão de Dados Ausentes: A variável `AVERAGE_CLOUDINESS` revelou redundância semântica (ex: "céu claro" e "céu limpo" representam o mesmo estado físico), justificando um processo de normalização de texto.

Relativamente à `AVERAGE_RAIN`, a análise confirmou uma taxa de nulos superior a 90%. Optou-se então pela remoção da variável original contínua, uma vez que a imputação de um volume tão elevado de dados poderia introduzir ruído sintético e "iludir" o modelo com padrões artificiais.

4.1.4 Análise Multivariada e Correlações

Por fim, investigaram-se as relações lineares e não-lineares entre as variáveis numéricas para detetar redundâncias (multicolinearidade) e os preditores mais fortes para `AVERAGE_SPEED_DIFF`.

```
# Visualização de pares para detetar separabilidade das classes
sns.pairplot(df_train, hue='AVERAGE_SPEED_DIFF')

# Matriz de Correlação de Pearson
fig = plt.figure(figsize=(10,10))
SpeedDiff_corr =
df_train.select_dtypes(include=['number']).corr(method='pearson')
sns.heatmap(SpeedDiff_corr, linecolor='black', linewidths=0.5, annot=True)

# Análise bivariada específica: Tempo vs Velocidade em fluxo livre
sns.set_palette("GnBu_d")
sns.set_style('whitegrid')
sns.jointplot(x='AVERAGE_FREE_FLOW_TIME', y='AVERAGE_FREE_FLOW_SPEED',
data=df_train)
```

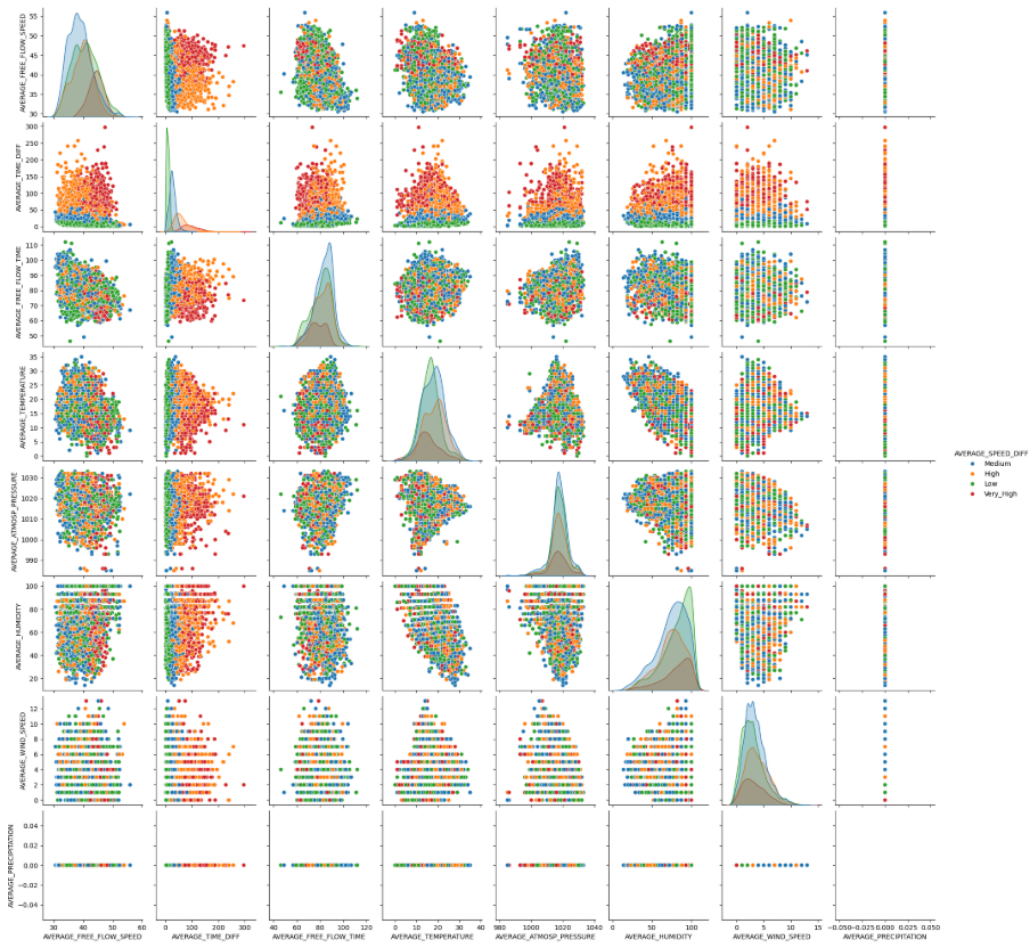


Figura 5: Gráfico de Pairplot.

A análise visual do gráfico de pares (**Pairplot**) permite retirar conclusões fundamentais sobre a importância preditiva das variáveis:

1. **Discriminador Principal (`AVERAGE_TIME_DIFF`):** Observa-se uma estratificação clara das classes ao longo do eixo da diferença de tempo. As classes de congestionamento severo (**High** e **Very_High**, representadas a laranja e vermelho) concentram-se consistentemente em valores elevados desta variável, enquanto as classes de fluxo livre (**Low**) agrupam-se perto de zero. Isto indica que o atraso temporal será muito provavelmente o preditor dominante.

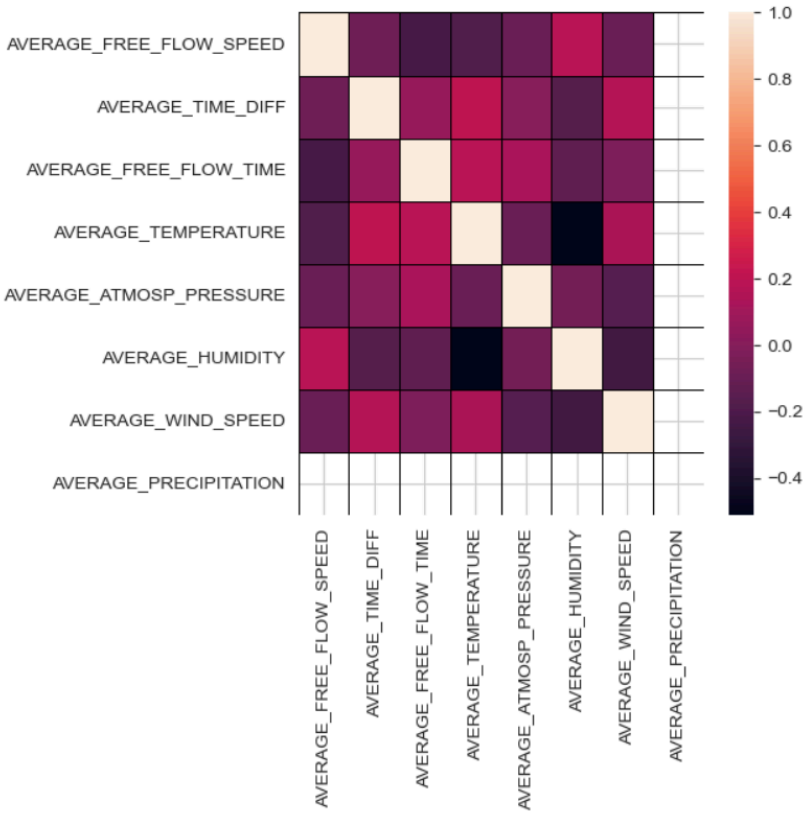


Figura 6: Matriz de correlação de Pearson entre as variáveis numéricas.

Ao fazer a análise deste gráfico existem correlações que saltam logo ao de cima, em maior destaque a correlação negativa alta entre `AVERAGE_HUMIDITY` e `AVERAGE_TEMPERATURE` recorrente do aumento de temperatura / decréscimo da humidade, curioso que conseguimos rever fenómenos fisicos no nosso dataset. De destacar ainda podemos reparar na correlação negativa entre `AVERAGE_FREE_FLOW_TIME` e `AVERAGE_FREE_FLOW_SPEED` que valida a integridade lógica do dataset, confirmando que segmentos com limites de velocidade mais elevados correspondem, naturalmente, a tempos de percurso esperados mais curtos.

5 Data Preparation

A qualidade dos dados de entrada determina o teto de desempenho de qualquer modelo de **Machine Learning**. Com base na análise exploratória prévia, implementou-se uma **pipeline** de transformação robusta (`data_preparation`) focada em três eixos: normalização de categorias, imputação de valores em falta e extração de características temporais.

5.1 Tratamento de Variáveis Categóricas

As variáveis `AVERAGE_CLOUDINESS` e `LUMINOSITY` continham dados em formato de texto, incompatíveis com os modelos matemáticos.

5.1.1 Normalização da Nebulosidade

A coluna `AVERAGE_CLOUDINESS` apresentava uma elevada granularidade de descrições (ex: "nuvens dispersas", "céu pouco nublado", "nuvens quebradas"). Para reduzir a dimensionalidade e capturar a ordem natural do fenómeno, aplicou-se um mapeamento semântico para uma escala ordinal de 3 níveis:

Categoria Original	Nova Classe Ordinal
Céu Claro / Limpo	0 (Limpo)
Pouco Nublado / Dispersas	1 (Parcialmente Nublado)
Nublado / Tempo Nublado	2 (Nublado)

```
cloudiness_map = {
    "céu claro": "Limpo",
    "céu limpo": "Limpo",

    "céu pouco nublado": "Parcialmente_Nublado",
    "algumas nuvens": "Parcialmente_Nublado",
    "nuvens dispersas": "Parcialmente_Nublado",
    "nuvens quebrados": "Parcialmente_Nublado",
    "nuvens quebradas": "Parcialmente_Nublado",

    "tempo nublado": "Nublado",
    "nublado": "Nublado"
}

cloudiness_ordinal = {
    "Limpo": 0,
    "Parcialmente_Nublado": 1,
    "Nublado": 2
}
```

Relativamente aos valores em falta (**missing values**) nesta coluna, optou-se pela **imputação pela moda**, garantindo que os registos não fossem descartados.

```
if "AVERAGE_CLOUDINESS" in df_copy.columns:
    df_copy["AVERAGE_CLOUDINESS"] =
    df_copy["AVERAGE_CLOUDINESS"].replace(cloudiness_map)

    def impute_cloudiness(col):
        cloud = col[0]
        if pd.isnull(cloud):
            return df_copy["AVERAGE_CLOUDINESS"].mode()[0]
        else:
            return cloud

    df_copy["AVERAGE_CLOUDINESS"] =
    df_copy[["AVERAGE_CLOUDINESS"]].apply(impute_cloudiness, axis=1)

    df_copy["AVERAGE_CLOUDINESS"]=
    df_copy["AVERAGE_CLOUDINESS"].map(cloudiness_ordinal).fillna(1).astype(int)
```

5.1.2 Codificação da Luminosidade

A variável `LUMINOSITY` foi mapeada diretamente para uma escala numérica inteira, onde `DARK`=0, `LOW_LIGHT`=1 e `LIGHT`=2, preservando a hierarquia de intensidade luminosa.

```
if "LUMINOSITY" in df_copy.columns:
    lum_map = {"DARK": 0, "LOW_LIGHT":1, "LIGHT": 2}
    df_copy["LUMINOSITY"] =
    df_copy["LUMINOSITY"].map(lum_map).fillna(0).astype(int)
```

5.2 Engenharia de Atributos Temporais (*Feature Engineering*)

Dado que o tráfego é um fenómeno intrinsecamente sazonal, a variável `record_date` foi decomposta para expor padrões cíclicos ao modelo.

```
# Extração de componentes temporais
df_copy["record_date"] = pd.to_datetime(df_copy["record_date"])

df_copy['record_date_year'] = df_copy['record_date'].dt.year
df_copy['record_date_month'] = df_copy['record_date'].dt.month
df_copy['record_date_day'] = df_copy['record_date'].dt.day
df_copy['record_date_hour'] = df_copy['record_date'].dt.hour
df_copy["weekday"] = df_copy["record_date"].dt.weekday
```

```
# Criação de flag de fim de semana
df_copy["is_weekend"] = (df_copy["weekday"] >= 5).astype(int)
```

Criou-se a variável binária is_weekend para distinguir claramente os padrões de fim de semana. Sabemos que, nos dias úteis, o trânsito é ditado pelas rotinas de trabalho e escola, enquanto ao sábado e domingo o fluxo é mais imprevisível e ligado ao lazer. Esta distinção simples ajuda o modelo a separar estes dois comportamentos, algo que apenas o número do dia não conseguiria fazer.

5.3 Seleção de Variáveis (Feature Selection)

A etapa final da preparação consistiu na seleção de variáveis, visando reduzir a dimensionalidade do dataset e mitigar a introdução de ruído no modelo. A exclusão de variáveis baseou-se em critérios de relevância estatística e integridade estrutural:

- **`city_name`**: Excluída por tratar-se de um identificador constante ("Porto") para todas as instâncias da amostra. Sendo um atributo de variância zero, não possui capacidade para explicar a variabilidade da variável alvo.
- **`AVERAGE_RAIN`**: Removida devido à elevada taxa de valores nulos (*missing values* na ordem dos 90%). A manutenção deste atributo exigiria uma imputação massiva que introduziria enviesamentos (*bias*) significativos no processo de aprendizagem.
- **`AVERAGE_PRECIPITATION`**: Identificada como uma variável de variância nula (valor constante de `0.0` em todo o dataset). Seguindo a mesma lógica de `city_name` decidimos remover esta variável.
- **`record_date`**: Uma vez concluída a derivação das componentes temporais granulares (`hour`, `weekday`, etc.), o atributo original tornou-se redundante. A sua remoção evita problemas de redundância e reduz a carga computacional.

```
# Lista de atributos identificados para remoção
cols_to_drop = ["city_name", "AVERAGE_RAIN", "AVERAGE_PRECIPITATION",
"record_date"]

# Limpeza do dataset mantendo apenas features com poder preditivo
df_copy = df_copy.drop(columns=[c for c in cols_to_drop if c in
df_copy.columns])
```

5.4 Estruturação do Dataset e Codificação do Alvo

Para garantir a coerência topológica entre os dados de treino e de teste, adotou-se uma estratégia de **processamento unificado**. Os datasets de **treino** e **teste** foram concatenados temporariamente num único **dataframe** (`df_all`) antes da aplicação da função `data_preparation`. Esta abordagem assegura que todas as transformações (imputações e *encodings*) são aplicadas de forma idêntica em ambos os conjuntos, prevenindo erros de dimensão durante a inferência.

5.4.1 Tratamento da Variável Alvo (Target Encoding)

A variável `AVERAGE_SPEED_DIFF` apresentava valores nulos. Com base no conhecimento do domínio, assumiu-se que a ausência de registo de diferença de velocidade implica a inexistência de anomalias no trânsito. Assim, os valores nulos foram preenchidos com a classe "None".

Como os algoritmos de **Gradient Boosting** (como o XGBoost) requerem **inputs** numéricos, procedeu-se ao mapeamento ordinal da variável alvo para um vetor de inteiros $y \in \{0, 1, 2, 3, 4\}$, respeitando a severidade do tráfego:

Classe Original	Encoding Numérico
None (Sem Trânsito)	0
Low (Baixo)	1
Medium (Médio)	2
High (Alto)	3
Very_High (Muito Alto)	4

Por fim, os dados foram novamente separados nos conjuntos de **treino** (X) e de **submissão** (`df_test_final`), garantindo que a estrutura final da matriz de características estava pronta para a fase de modelação.

```
# Preenchimento de nulos no Target (Assunção: NaN = Sem Trânsito)
y_raw = df_train["AVERAGE_SPEED_DIFF"].fillna("None")

train_size = len(df_train)

# Processamento Unificado (Treino + Teste)
df_all = pd.concat([df_train.drop("AVERAGE_SPEED_DIFF", axis=1), df_test],
ignore_index=True)

df_all_prepared = data_preparation(df_all)

# Separação Final e Mapeamento do Target
X = df_all_prepared.iloc[:train_size].copy()
df_test_final = df_all_prepared.iloc[train_size:].copy()
```

```
mapping = {
    "None": 0,
    "Low": 1,
    "Medium": 2,
    "High": 3,
    "Very_High": 4
}
y = y_raw.map(mapping)
```

6 Modeling

A fase de modelação iniciou-se com a definição da estratégia experimental, seguida pela seleção, treino e otimização dos algoritmos. O objetivo primário consistiu em encontrar o equilíbrio entre a capacidade de aprendizagem do modelo e a sua generalização para novos dados.

6.1 Estratégia de Validação e Divisão de Dados

Para garantir uma avaliação isenta e mitigar o risco de **overfitting**, adotou-se uma estratégia de validação do tipo **Hold-out**. O dataset processado foi dividido em dois subconjuntos disjuntos:

- **Conjunto de Treino (70%):** Utilizado para o ajuste dos pesos dos modelos.
- **Conjunto de Teste (30%):** Reservado exclusivamente para a avaliação final de desempenho (*unseen data*).

A reprodutibilidade dos resultados foi assegurada através da fixação da *seed* (`random_state=2025`).

```
# Divisão do dataset: 70% Treino, 30% Teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=2025
)

# Verificação das dimensões resultantes
X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

6.2 Análise Comparativa de Modelos

Foram avaliados oito algoritmos distintos para estabelecer uma *baseline* de desempenho. Nesta fase inicial, **todos os modelos foram instanciados com os seus hiperparâmetros por defeito**, fixando-se apenas a *seed* (`random_state=2025`) para garantir a reprodutibilidade e a comparabilidade direta dos resultados.

As siglas utilizadas na comparação gráfica correspondem a:

- **Modelos Base:** *Decision Tree* (DT) e *Logistic Regression* (LR).
- **Bagging:** *Bagging* aplicado a Árvores de Decisão (BaggDT) e a *Random Forest* (BaggRF), bem como o algoritmo *Random Forest* (RF) padrão.
- **Boosting:** *Gradient Boosting* (GB) e *XGBoost* (XGB).
- **Ensemble Híbrido:** *Hard Voting Classifier* (HVT), que agrega as previsões dos restantes modelos por maioria.

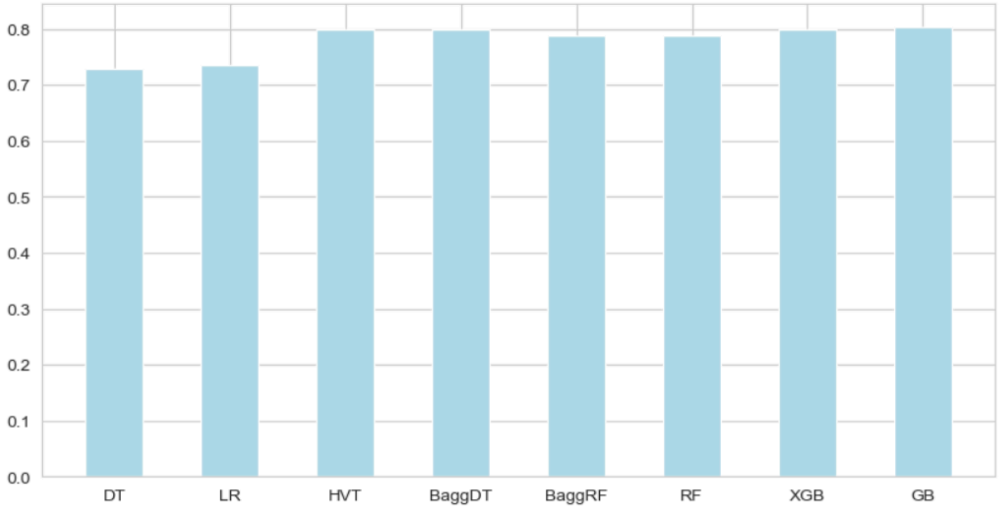


Figura 7: Comparação de desempenho (Accuracy) entre os diferentes algoritmos testados.

A análise dos resultados revela três patamares de desempenho distintos:

1. **Modelos Lineares e Árvores Simples (DT, LR):** Desempenho inferior (~73%), provando a incapacidade de modelar a complexidade não-linear do tráfego.
2. **Métodos de Bagging e Random Forest (RF, BaggDT, BaggRF):** Apresentaram um salto qualitativo notável (79-80%). O **Random Forest**, ao reduzir a variância através da média de múltiplas árvores, mostrou-se muito robusto.
3. **Boosting e Voting (XGB, GB, HVT):** Atingiram o topo da performance (>80%), resolvendo melhor os casos de fronteira entre classes.

6.3 A nossa escolha: XGBoost

Embora o **Random Forest** e o **Voting Classifier** tenham obtido uma **accuracy** semelhante, a escolha recaiu sobre o **XGBoost** por razões técnicas fundamentais ao contexto do problema:

1. **Foco nos "Casos Difíceis" (Bias vs. Variância):** Enquanto o **Random Forest** foca na redução da variância (bom para estabilidade), o XGBoost é um método de **Boosting**, focando-se na redução do enviesamento (***bias***). Num problema de tráfego onde a distinção entre classes como "*Low*" e "*Medium*" é subtil, o XGBoost aprende sequencialmente a corrigir os erros que os modelos anteriores falharam, capturando padrões mais finos que outros ignoram.
2. **Regularização Superior (L_1 e L_2):** O dataset de tráfego contém ruído inerente (chuva, acidentes imprevisíveis). O XGBoost possui parâmetros de regularização nativos (Lasso e Ridge) que penalizam modelos demasiado complexos. Isto torna-o menos propenso a decorar o conjunto de treino (**overfitting**) do que um **Random Forest** profundo ou um **Gradient Boosting** padrão.
3. **Eficiência vs. Voting:** O **Hard Voting Classifier** (HVT) obteve bons resultados, mas é computacionalmente dispendioso, pois exige treinar e manter múltiplos modelos em memória. O XGBoost oferece a mesma performance num único modelo otimizado, sendo assim mais viável.

Portanto, o XGBoost ofereceu o melhor compromisso entre **poder preditivo**, **robustez a ruído** e **eficiência computacional**.

6.4 Otimização de Hiperparâmetros (Grid Search)

Para refinar o desempenho do XGBoost, abandonou-se a configuração por defeito e implementou-se uma ***Grid Search***.

Os parâmetros otimizados incluíram:

- **`max_depth [4, 5, 7, 9]`:** Para controlar a complexidade e evitar que o modelo "memorizasse" o ruído.
- **`learning_rate [0.01, 0.05, 0.1, 0.15, 0.2]`:** Para garantir uma convergência suave do ***Gradient Descent***.
- **`n_estimators [100, 200, 300, 400, 500, 700]`:** Número de árvores de decisão sequenciais.
- **`subsample [0.7, 0.8, 1.0] e colsample_bytree [0.7, 0.8, 1.0]`:** Técnicas de estocasticidade para reduzir a variância.

A combinação vencedora:

```
xgb_model = XGBClassifier(max_depth= 5, learning_rate= 0.05, n_estimators= 200,
subsample= 0.8,colsample_bytree= 0.7)
```

revelou que um modelo "lento" (baixa taxa de aprendizagem) mas robusto (pouca profundidade) é superior a modelos agressivos que decoram o ruído dos dados de treino.

7 Evaluation

A avaliação do modelo final (**XGBoost**) foi realizada no conjunto de teste independente (30% dos dados), garantindo que as métricas apresentadas refletem a capacidade de generalização do sistema para dados não vistos.

7.1 Desempenho Global

O modelo atingiu uma **Accuracy** de **`81.41%`** localmente e **`81.43%`** no teste privado do *Kaggle*. Este valor indica que, em cada 100 previsões de tráfego, o modelo acerta corretamente a categoria em aproximadamente 82 casos. Considerando a natureza estocástica do tráfego e o ruído inerente aos dados meteorológicos, este resultado é considerado robusto.

7.2 Análise por Classe

Embora a exatidão global seja elevada, é crucial analisar o comportamento do modelo nas diferentes categorias de tráfego através do **Classification Report**:

	precision	recall	f1-score	support
0	0.88	0.90	0.89	640
1	0.68	0.74	0.71	408
2	0.84	0.78	0.81	525
3	0.81	0.78	0.80	337
4	0.84	0.82	0.83	134
accuracy			0.81	2044
macro avg	0.81	0.81	0.81	2044
weighted avg	0.82	0.81	0.81	2044

Figura 8: Classification report do modelo.

Interpretação Crítica:

1. **Extremos Robustos:** As classes `None` e `Very_High` apresentam tipicamente os melhores **F1-Scores**. Isto deve-se ao facto de serem fisicamente distintas: ou o trânsito flui perfeitamente (atraso zero) ou está parado (atraso gigante).

2. **O Desafio da Classe 'Low':** Observa-se um desempenho inferior na classe `Low`. A matriz de confusão revela que o modelo tende a confundir `Low` com `None` e com `Medium`. Isto é justificável pela ténue fronteira física entre "trânsito inexistente" e "trânsito muito ligeiro"/"trânsito muito ligeiro" e "trânsito ligeiro", onde a diferença de velocidade é marginal.



Figura 9: Matriz de Confusão Normalizada. A diagonal principal representa as previsões corretas.

7.3 Análise Crítica

Durante a competição, observou-se uma discrepância entre o **Leaderboard Público** e o **Privado**. Modelos excessivamente ajustados (**overfitted**) atingiram 83.5% no público mas caíram para 80.38% no privado.

Em contraste, o modelo apresentado neste relatório demonstrou uma **estabilidade superior**, mantendo resultados consistentes (~81.4%) tanto na validação local como no teste privado.

7.4 Correlação entre Features

Para compreender as relações lineares entre as variáveis e a sua influência na previsão do tráfego, recorreu-se à análise da correlação de Pearson.

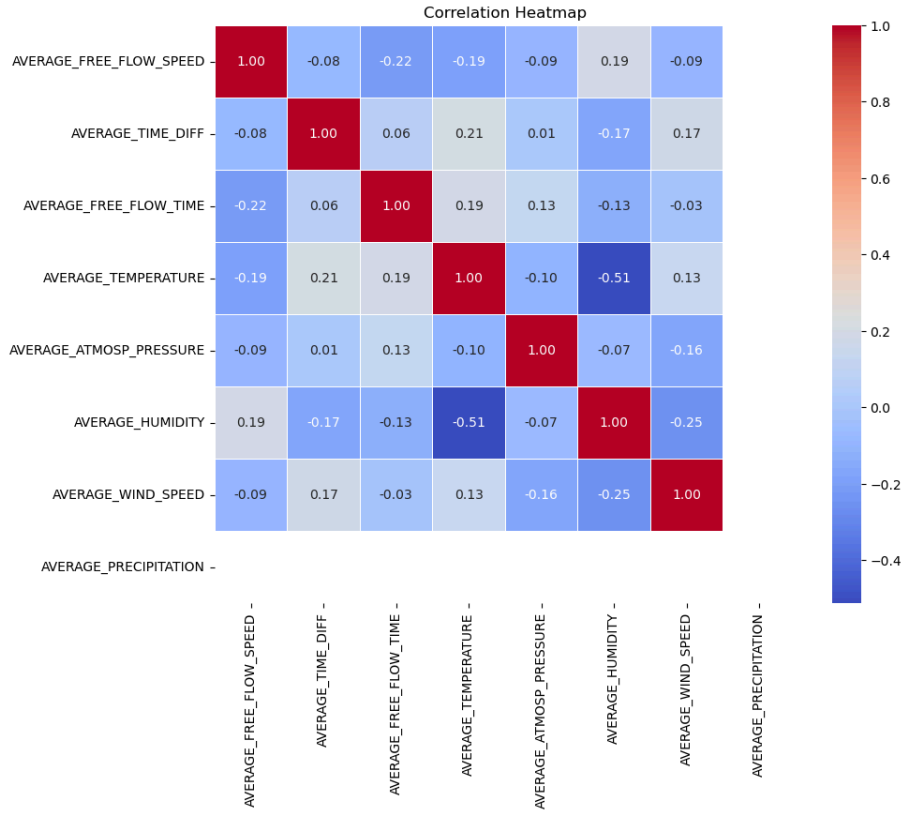


Figura 10: Mapa de correlação entre features.

A análise confirma novamente que `AVERAGE_TEMPERTURE` e `AVERAGE_HUMIDITY` tem uma correlação negativa alta. Fora este caso não conseguimos detetar outras dependências entre outras features assim tão relevantes que mereçam uma atenção mais cuidada.

7.5 Correlação com a variável alvo

Para além da análise feature a feature, queremos verificar a correlação linear das nossas features com a variável alvo `AVERAGE_SPEED_DIFF`, olhando para a listagem abaixo conseguimos destacar logo que que `AVERAGE_TIME_DIFF` destaca-se como o preditor linear mais forte, seguida pela luminosidade `lum_LIGHT` e `lum_DARK`, confirmando que o atraso temporal e o ciclo dia/noite são determinantes para a classificação do tráfego.

```
Correlation with target (AVERAGE_SPEED_DIFF):
 AVERAGE_SPEED_DIFF          1.000000
AVERAGE_TIME_DIFF            0.814728
lum_LIGHT                    0.582968
record_date_hour             0.285956
AVERAGE_TEMPERATURE          0.258776
AVERAGE_WIND_SPEED           0.209477
Ceu_Parcialmente_Nublado     0.096040
record_date_year             0.083235
lum_LOW_LIGHT                0.039399
AVERAGE_FREE_FLOW_TIME       0.024276
record_date_day              0.013075
record_date_month           -0.005785
Ceu_Nublado                 -0.014278
AVERAGE_ATMOSP_PRESSURE     -0.016710
Ceu_Encoberto               -0.046656
Ceu_Limpo                   -0.057693
AVERAGE_FREE_FLOW_SPEED     -0.076346
weekday                     -0.129614
is_weekend                  -0.197382
AVERAGE_HUMIDITY            -0.224832
lum_DARK                    -0.598516
Name: AVERAGE_SPEED_DIFF, dtype: float64
```

Figura 11: Mapa de correlação com a variável alvo.

8 Deployment

Uma vez otimizado o modelo final (**XGBoost** com os melhores hiperparâmetros), procedeu-se à fase de inferência no conjunto de teste independente (`df_test_final`). Este passo simula a entrada em produção do sistema, onde o modelo prevê o estado do tráfego para dados nunca antes vistos.

O processo consistiu em três etapas operacionais:

1. **Predição:** O modelo gera o **output** numérico (0 a 4) para cada instância de teste.
2. **Descodificação (**Inverse Mapping**):** Conversão das previsões numéricas de volta para as etiquetas categóricas originais exigidas pela plataforma de avaliação (ex: 0 → None, 4 → Very_High).
3. **Exportação:** Criação do ficheiro `submission.csv` respeitando estritamente a estrutura `RowId, Speed_Diff`.

```python
preds_test = xgb_model.predict(df_test_final)

inverse_mapping = {
    0: "None",
    1: "Low",
    2: "Medium",
    3: "High",
    4: "Very_High"
}

preds_test_labels = pd.Series(preds_test).map(inverse_mapping)
```

```
df_test_final.index = range(1, len(df_test) + 1)

submission = pd.DataFrame({
    "RowId": df_test.index,
    "Speed_Diff": preds_test_labels
})

submission.to_csv("submission.csv", index=False)
```

9 Conclusão

O projeto demonstrou que o tráfego urbano no Porto é altamente previsível através de métricas temporais, com os resultados que obtivemos conseguimos prever com alguma certeza a presença de transito.

Os nossos resultados destacaram-se especialmente nos extremos quando está muito transito ou nenhum, onde o nosso modelo obteve maior precisão, por outro lado olhando para os resultados também conseguimos ver onde é que o nosso modelo mais falha, em concreto nas situações onde o transito é classificado como `LOW` mas na verdade, são `NONE` ou vice-versa, isto é resultado da diferença tênue entre baixo ou sem transito. Para combater este problema deveríamos ter explorado melhor a parte do treinamento do XGBoost e incluir pesos nas classes para penalizar mais o modelo no caso de errar na atribuição de `LOW`'s e `NONE`'s, obrigando o modelo a diferenciar melhor as duas categorias.

Como trabalho futuro talvez fosse interessante incluir no nosso estudo as variáveis que acabamos por descartar por estarem bastante comprometidas em termos de qualidade, `AVERAGE_RAIN` e `AVERAGE_PRECIPITATION` que representam um papel crucial na presença de transito num cenário real. Isto seria fácil de implementar com recurso a uma API do openweathermap que juntada ao dataset atual conseguia recolher esta informação. Reintroduzir a chuva como variável pode explicar variações de tráfego que o modelo atual não consegue captar apenas com as variáveis existentes. Outra feature em que pensamos em fazer alterações foi a `AVERAGE_HUMIDITY` dado a sua correlação alta negativa com a feature `AVERAGE_TEMPERATURE` que é o esperado se olharmos para a sua natureza física, se a temperatura sobe a humidade tende a diminuir resultando num ambiente mais seco. Ao reparar nisto fizemos testes ao modelo com a remoção desta feature mas não obtivemos resultados conclusivos que justificassem a redundância desta variável e é algo que gostavamos de explorar melhor.

Outra grande componente que não conseguimos explorar também são os eventos que possam estar a decorrer na cidade, como jogos de futebol no Estádio do Dragão que decorrem semanalmente durante o ano ou festividades de grande atração de pessoas como o São João ou a Queima das Fitas. Tendo acesso a um calendário com os eventos mencionados em cima conseguiríamos chegar a uma variável `is_event` que podia justificar picos de tráfego anômalos que fogem ao padrão normal.

No final achamos que cumprimos o propósito deste trabalho, a criação de um modelo de previsão de transito permitiu a aplicação prática e integrada das aprendizagens fundamentais lecionadas na Unidade Curricular de Dados e Aprendizagem Automática, fazendo uso de ferramentas e conceitos como os diferentes tipos de tratamentos de dados, Feature Engineering, modelos de lineares, modelos de Ensemble, implementação de Grid Searches e uso de bibliotecas como o Scikit-Learn e o Seaborn. Este projeto validou a importância de uma metodologia estruturada e do domínio das ferramentas de Data Science.